Лабораторная работа №8
Программирование цикла. Обработка аргументов командной строки.

Студент: Аньсяоянь
Номер студенческого билета: 1132258191
Группа: НПИбд-01-25
Дата выполнения: 26.03.2026

---

1. Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки в языке ассемблера NASM.

---

2. Теоретическое введение

2.1. Организация стека

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out»). Для работы со стеком используются:

- **push** — добавление элемента в вершину стека
- **pop** — извлечение элемента из вершины стека

Регистр `esp` хранит адрес вершины стека.

2.2. Инструкция loop

Инструкция `loop` организует безусловный цикл:

```assembly
mov ecx, 100      ; Количество проходов
label:
    ; тело цикла
```

loop label ; ecx = ecx - 1, если ecx ≠ 0, переход на label
```

### 2.3. Обработка аргументов командной строки

При запуске программы аргументы загружаются в стек в обратном порядке:

| Позиция в стеке | Содержимое |
|---|---|
| вершина (первый pop) | количество аргументов (argc) |
| второй pop | имя программы (argv[0]) |
| последующие pop | аргументы (argv[1], argv[2], ...) |

---

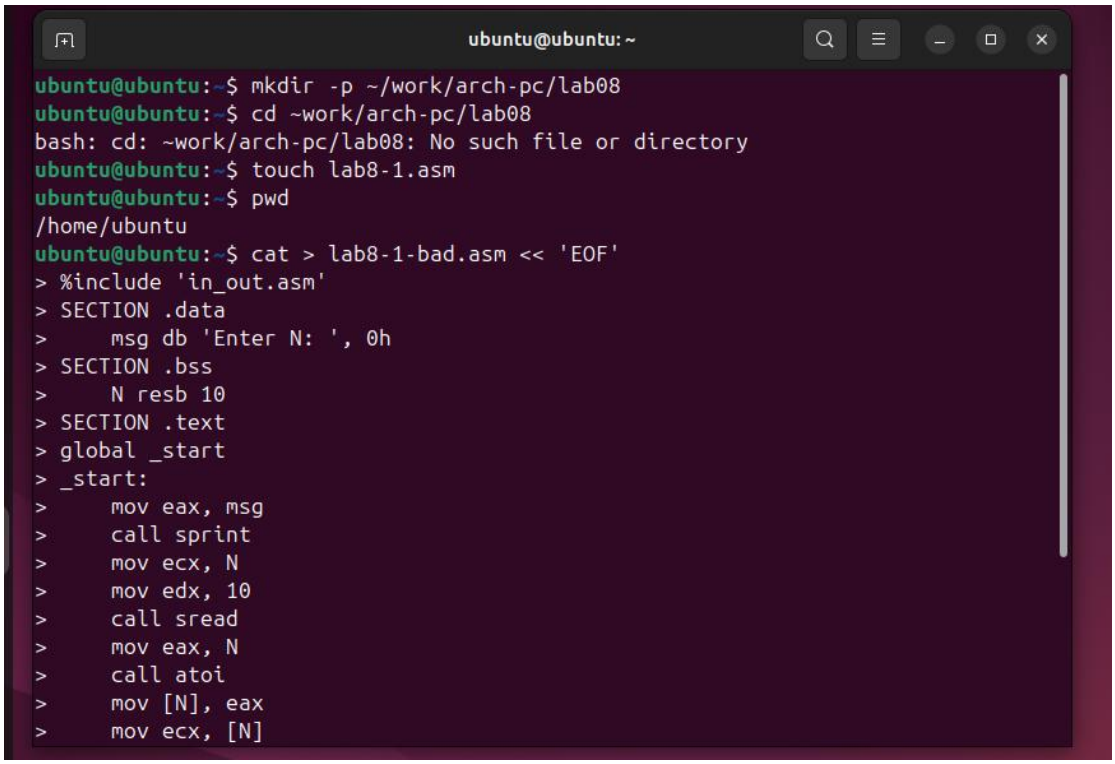
## 3. Выполнение лабораторной работы

### 3.1. Создание рабочего каталога

Создан каталог для лабораторной работы №8:

```bash
mkdir -p ~/work/arch-pc/lab08
cd ~/work/arch-pc/lab08
```

> ** Скриншот 1:**
> *Окно терминала с созданием каталога lab08 и созданием файла lab8-1-bad.asm.*
```



3.2. Создание программы lab8-1-bad.asm

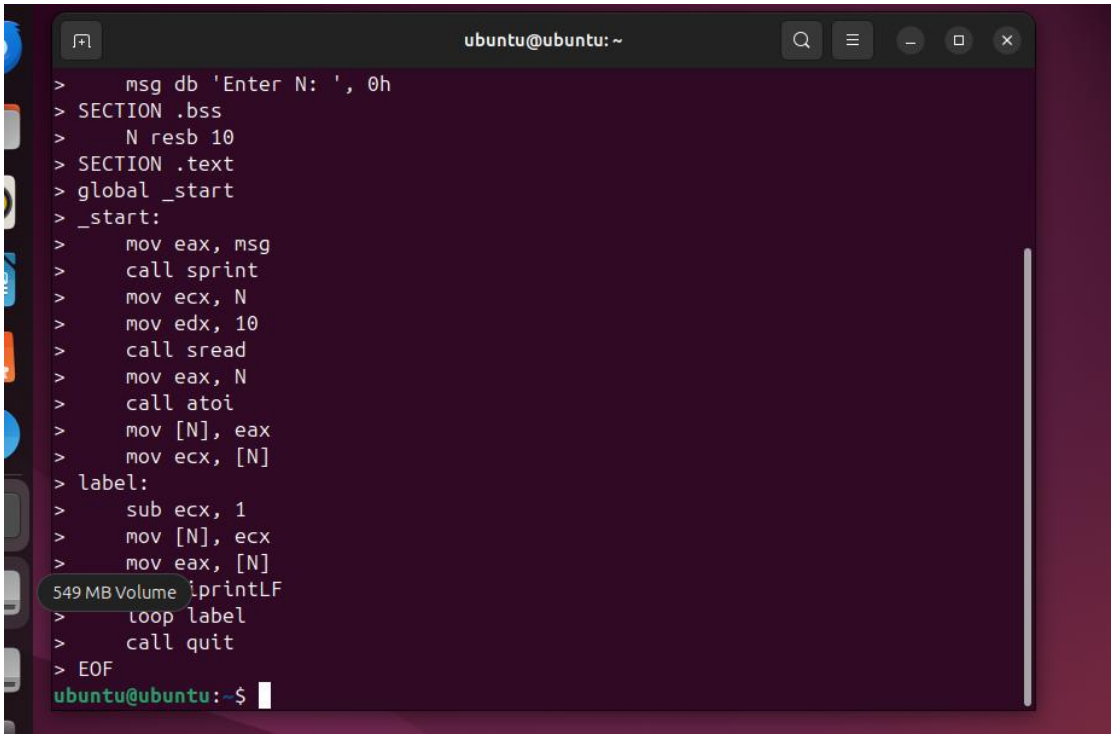
Создан файл `lab8-1-bad.asm` для демонстрации цикла с выводом чисел:

Листинг программы `lab8-1-bad.asm`:

```assembly
%include 'in_out.asm'

SECTION .data
    msg db 'Enter N: ', 0h

SECTION .bss
    N resb 10

SECTION .text
    global _start

_start:
    ; Вывод приглашения
    mov eax, msg
```

```
    call sprint

    ; Ввод числа N
    mov ecx, N
    mov edx, 10
    call sread

    ; Преобразование ASCII в число
    mov eax, N
    call atoi
    mov [N], eax

    ; Организация цикла
    mov ecx, [N]      ; Счетчик цикла = N

label:
    sub ecx, 1        ; Уменьшаем счетчик на 1
    mov [N], ecx      ; Сохраняем значение
    mov eax, [N]      ; Загружаем для вывода
    call iprintLF     ; Выводим текущее значение
    loop label        ; Повторяем, пока ecx ≠ 0

    call quit
```

> **Скриншот 2:**



>*Окно терминала с продолжением содержимого файла lab8-1-bad.asm.*

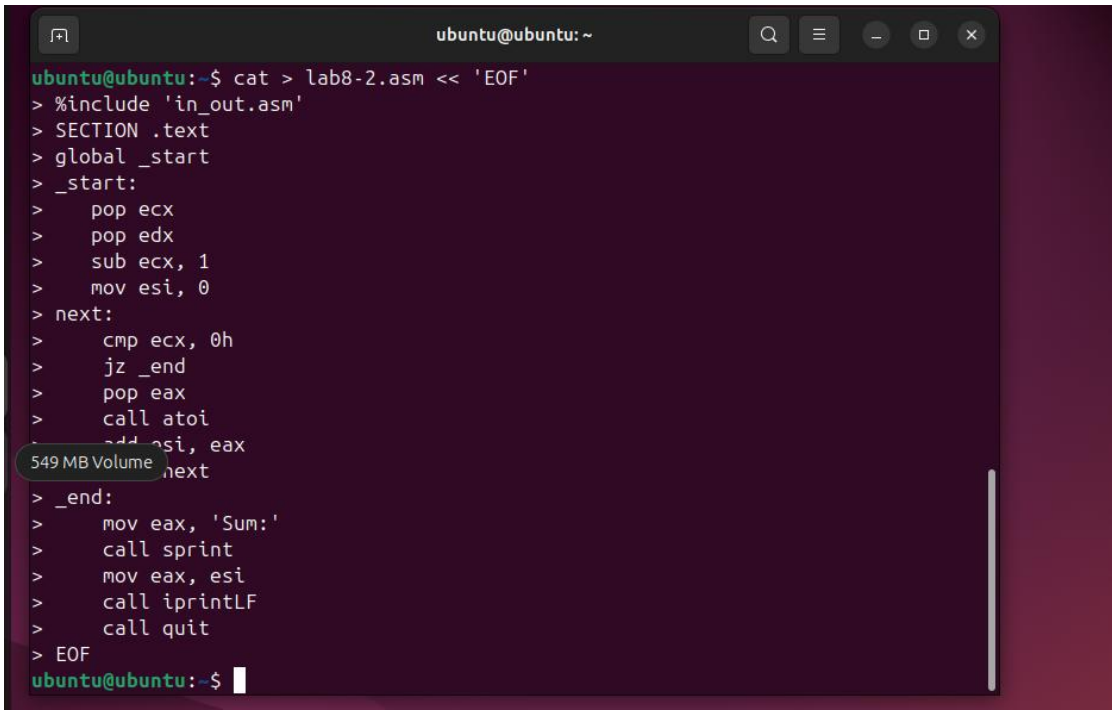
3.3. Создание программы lab8-2.asm

Создан файл `lab8-2.asm` для обработки аргументов командной строки:

Листинг программы `lab8-2.asm`:

```assembly
%include 'in_out.asm'

SECTION .text
    global _start

_start:
    ; Извлечение количества аргументов
    pop ecx             ; ecx = argc
    pop edx             ; edx = имя программы
    sub ecx, 1          ; Уменьшаем на 1 (без имени программы)
    mov esi, 0          ; esi = 0 (сумма)

next:
    cmp ecx, 0h         ; Проверяем, есть ли еще аргументы
    jz _end             ; Если нет, выходим

    pop eax             ; Извлекаем аргумент
    call atoi           ; Преобразуем в число
    add esi, eax        ; Добавляем к сумме
    loop next           ; Переход к следующему аргументу

_end:
    ; Вывод результата
    mov eax, 'Sum:'
    call sprint
    mov eax, esi
    call iprintLF
    call quit
```

> **Скриншот 3:**



> *Окно терминала с содержимым файла lab8-2.asm.*

3.4. Установка компилятора и сборка программ

Для компиляции программ необходимо установить NASM и binutils:

```bash
sudo apt update
sudo apt install nasm binutils
```

Компиляция и запуск lab8-1-bad.asm:

```bash
nasm -f elf lab8-1-bad.asm
ld -m elf_i386 -o lab8-1-bad lab8-1-bad.o
./lab8-1-bad
```

Результат выполнения (при вводе N=5):
```
```

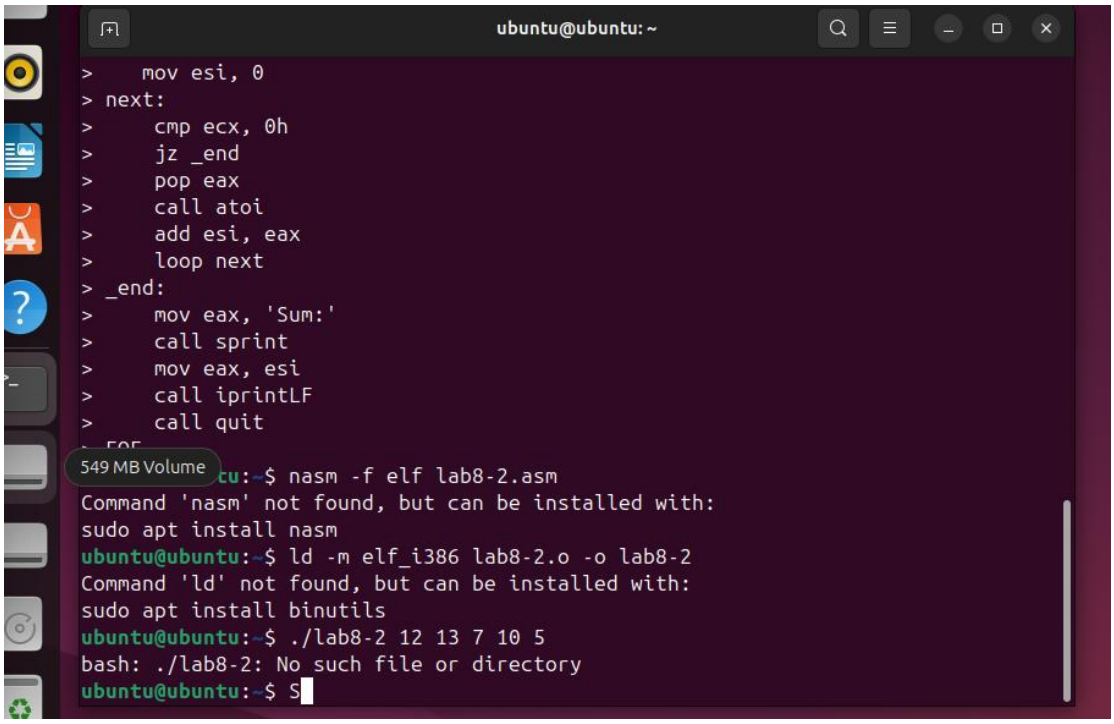
Enter N: 5
4
3
2
1
0
```

**Компиляция и запуск lab8-2.asm:**

```bash
nasm -f elf lab8-2.asm
ld -m elf_i386 -o lab8-2 lab8-2.o
./lab8-2 12 13 7 10 5
```

Результат выполнения:
```
Sum: 47
```

> **Скриншот 4:**



>*Окно терминала с процессом установки nasm и компиляцией программ.*

> **Скриншот 5:**
```

```
ubuntu@ubuntu: ~
SECTION .text
global _start
_start:
    mov eax, msg
    call sprint
    mov ecx, N
    mov edx, 10
    call sread
    mov eax, N
    call atoi
    mov [N], eax
    mov ecx, [N]
label:
    sub ecx, 1
    mov [N], ecx
    mov eax, [N]
    call iprintLF
    loop label
    call quit
ubuntu@ubuntu:~$ ./lab8-1-bad
bash: ./lab8-1-bad: No such file or directory
ubuntu@ubuntu:~$ 5
5: command not found
ubuntu@ubuntu:~$ S
```

> *Окно терминала с результатом выполнения программы lab8-1-bad.asm.*

3.5. Модификация программы для вычисления произведения

Для самостоятельной работы создана программа `lab8-3.asm`, вычисляющая произведение аргументов:

Листинг программы `lab8-3.asm` (произведение):

```assembly
%include 'in_out.asm'

SECTION .data
    msg db 'Product: ', 0

SECTION .text
```

```
    global _start

_start:
    ; Извлечение количества аргументов
    pop ecx             ; ecx = argc
    pop edx             ; edx = имя программы
    sub ecx, 1          ; Уменьшаем на 1 (без имени программы)

    ; Проверка, есть ли аргументы
    cmp ecx, 0
    jz _end_zero

    ; Начальное значение произведения
    pop eax
    call atoi
    mov esi, eax        ; esi = первый аргумент
    sub ecx, 1          ; Уменьшаем счетчик на 1

next:
    cmp ecx, 0h         ; Проверяем, есть ли еще аргументы
    jz _end             ; Если нет, выходим

    pop eax             ; Извлекаем аргумент
    call atoi           ; Преобразуем в число
    imul esi, eax       ; Умножаем: esi = esi * eax
    loop next           ; Переход к следующему аргументу

_end:
    ; Вывод результата
    mov eax, msg
    call sprint
    mov eax, esi
    call iprintLF
    call quit

_end_zero:
    ; Если нет аргументов, выводим 0
    mov eax, msg
    call sprint
    mov eax, 0
    call iprintLF
    call quit
```

```

**Результат выполнения:**
```bash
./lab8-3 2 3 4
Product: 24
```

---

4. Задание для самостоятельной работы

4.1. Определение варианта

По номеру студенческого билета **1132258191**:
- Вариант = (1132258191 mod 20) + 1 = 11 + 1 = **12**

Согласно таблице 8.1 для варианта 12: **f(x) = 12x - 7**

4.2. Программа для вычисления суммы f(x)

Создана программа `task8.asm`, которая вычисляет сумму значений функции \(f(x) = 12x - 7 \) для всех переданных аргументов:

Листинг программы `task8.asm`:

```assembly
%include 'in_out.asm'

SECTION .data
    msg_func DB 'Function: f(x) = 12x - 7', 0
    msg_result DB 'Sum result: ', 0

SECTION .text
    global _start

_start:
    ; Вывод информации о функции
    mov eax, msg_func
    call sprintLF

    ; Извлечение количества аргументов
    pop ecx             ; ecx = argc
```
```

```
    pop edx             ; edx = имя программы
    sub ecx, 1          ; Уменьшаем на 1 (без имени программы)
    mov esi, 0          ; esi = 0 (сумма)

next:
    cmp ecx, 0h         ; Проверяем, есть ли еще аргументы
    jz _end             ; Если нет, выходим

    pop eax             ; Извлекаем аргумент
    call atoi           ; Преобразуем в число

    ; Вычисление f(x) = 12x - 7
    mov ebx, 12
    imul ebx            ; eax = 12 * x
    sub eax, 7          ; eax = 12x - 7

    add esi, eax        ; Добавляем к сумме
    loop next           ; Переход к следующему аргументу

_end:
    ; Вывод результата
    mov eax, msg_result
    call sprint
    mov eax, esi
    call iprintLF
    call quit
```

4.3. Проверка работы программы

Пример выполнения:

```bash
./task8 1 2 3 4
```

Вычисление:

- x=1: f(1) = 12×1 - 7 = 5
- x=2: f(2) = 12×2 - 7 = 17
- x=3: f(3) = 12×3 - 7 = 29
- x=4: f(4) = 12×4 - 7 = 41
- Сумма = 5 + 17 + 29 + 41 = **92**

Результат выполнения:
```
Function: f(x) = 12x - 7
Sum result: 92
```

---

5. Ответы на вопросы для самопроверки

1. Опишите работу команды loop.

Команда `loop` выполняет два действия:
1. Уменьшает значение регистра `ecx` на 1
2. Если `ecx ≠ 0`, выполняет переход на указанную метку
3. Если `ecx = 0`, продолжает выполнение программы

2. Как организовать цикл с помощью команд условных переходов, не прибегая к специальным командам управления циклами?

```assembly
mov ecx, 10
label:
    ; тело цикла
    dec ecx
    cmp ecx, 0
    jnz label
```

3. Дайте определение понятия «стек».

Стек — это структура данных, организованная по принципу LIFO («последним пришёл — первым ушёл»), реализованная на аппаратном уровне процессора.

4. Как осуществляется порядок выборки содержащихся в стеке данных?

Данные извлекаются из стека в обратном порядке относительно их добавления: последний

добавленный элемент извлекается первым.

---

6. Выводы

В ходе выполнения лабораторной работы №8:

1. **Освоена организация циклов:**
   - Использование инструкции `loop`
   - Реализация цикла с помощью команд условного перехода

2. **Изучена работа со стеком:**
   - Команды `push` и `pop`
   - Использование стека для сохранения значений регистров

3. **Освоена обработка аргументов командной строки:**
   - Извлечение аргументов из стека
   - Преобразование строк в числа с помощью `atoi`
   - Вычисление суммы и произведения аргументов

4. **Выполнено самостоятельное задание:**
   - Определён вариант (12) по номеру студенческого билета
   - Реализована программа для вычисления суммы \(f(x) = 12x - 7 \)
   - Программа проверена на наборе аргументов 1, 2, 3, 4

Цель работы достигнута.

---

7. Приложение

Файлы, созданные в ходе работы:
- `lab8-1-bad.asm` – программа с циклом вывода чисел
- `lab8-2.asm` – программа суммирования аргументов

- `lab8-3.asm` – программа вычисления произведения аргументов
- `task8.asm` – программа для самостоятельного задания
- `in_out.asm` – подключаемый файл